# CI/CD Interview Bible

From Fundamentals to Senior-Level Mastery

DevOps • SRE • Cloud Operations

A complete reference — concepts, tools, integrations,
troubleshooting, common errors, tricky questions & expected Q&A
for candidates with 5+ years of experience

# Table of Contents

# 1. CI/CD Fundamentals

## 1.1 What is CI/CD?

**Continuous Integration (CI)** is the practice of merging all developers' working copies to a shared mainline several times a day, with every merge triggering an automated build and test cycle. The goal is to detect integration bugs as early as possible.

**Continuous Delivery (CD)** extends CI so that every change that passes automated tests is automatically prepared for a release to production — but the actual production deployment still requires a manual approval/trigger.

**Continuous Deployment (CD)** goes one step further: every change that passes all pipeline stages is deployed to production automatically, with no human gate.

| Term | Build | Test | Release Prep | Deploy to Prod |
|---|---|---|---|---|
| Continuous Integration | Automatic | Automatic | - | Manual/Not included |
| Continuous Delivery | Automatic | Automatic | Automatic | Manual approval gate |
| Continuous Deployment | Automatic | Automatic | Automatic | Fully automatic |

## 1.2 Why CI/CD Matters

- Shorter feedback loops → bugs caught within minutes, not weeks.
- Smaller batch sizes → each change is easier to reason about and roll back.
- Consistent, repeatable, auditable release process — removes "works on my machine".
- Enables business agility: faster time-to-market, higher deployment frequency (a key DORA metric).
- Reduces manual toil for Ops/SRE, freeing time for reliability work.

## 1.3 The Four Key DORA Metrics Senior favorite

| Metric | What it measures | Elite performer benchmark |
|---|---|---|
| Deployment Frequency | How often code ships to production | On-demand, multiple times per day |
| Lead Time for Changes | Commit to production time | Less than 1 hour |
| Change Failure Rate | % of deployments causing a failure in production | 0-15% |
| Mean Time to Restore (MTTR) | Time to recover from a failed deployment/incident | Less than 1 hour |

> Interviewers at senior/SRE level frequently ask you to map pipeline design decisions back to these four metrics — always tie your answer to "this improves deployment frequency" or "this reduces change failure rate".

## 1.4 Anatomy of a Pipeline

A typical pipeline is a sequence of **stages**, each containing one or more **jobs/steps** that can run sequentially or in parallel:

```
Source → Build → Unit Test → Static Analysis (SAST/Lint) → Package/Artifact
   → Deploy to Dev → Integration/Contract Tests → Deploy to Staging
   → Security/DAST Scan → Manual Approval → Deploy to Prod → Smoke Test → Monitor
```

## 1.5 Core Principles

- **Single source of truth** — one artifact is built once and promoted through environments (build once, deploy many), never rebuilt per environment.
- **Fail fast** — cheapest/fastest checks (lint, unit tests) run first; expensive checks (E2E, performance) run later.
- **Idempotency** — re-running a pipeline/deployment produces the same end state.
- **Immutable infrastructure & artifacts** — never patch a running artifact; build a new one and replace it.
- **Pipeline as Code** — pipeline definitions live in version control (Jenkinsfile, .gitlab-ci.yml, YAML workflows), reviewed like application code.
- **Shift-left** — move testing, security, and quality checks as early as possible in the pipeline.
- **Everything is versioned** — app code, infra code, pipeline code, configuration.

## 1.6 CI vs CD vs DevOps vs SRE — How They Relate

CI/CD is a *practice/toolchain*. DevOps is the broader *culture and set of practices* that breaks down the silo between Dev and Ops, of which CI/CD is a key enabler. SRE is a specific *implementation philosophy* of DevOps (coined at Google) that applies software engineering discipline to operations problems, using SLOs/error budgets to decide how aggressively CI/CD should push changes — if the error budget is burnt, releases can be frozen even though the pipeline is technically "green".

# 2. Version Control & Branching Strategies

## 2.1 Branching Models

| Model | Description | Best for |
|---|---|---|
| Git Flow | Long-lived `develop` + `main`, feature/release/hotfix branches | Scheduled/versioned releases (mobile apps, on-prem software) |
| GitHub Flow | Single `main` branch, short-lived feature branches, PR + merge, deploy immediately | Web apps / SaaS with continuous deployment |
| Trunk-Based Development | Everyone commits to `trunk/main` frequently (at least daily), feature flags hide incomplete work | High-performing CI/CD teams, elite DORA performers |
| GitLab Flow | Combines feature branches with environment branches (e.g. `staging`, `production`) | Teams needing environment-based promotion with GitLab |

## 2.2 Trunk-Based Development & Feature Flags Senior favorite

Trunk-based development is considered a prerequisite for elite CI/CD performance because long-lived branches cause large, risky merges. Feature flags (LaunchDarkly, Unleash, Flagsmith, or home-grown) let teams merge incomplete code to trunk behind a flag, decoupling **deployment** (code reaches production) from **release** (code is exposed to users). This is a key distinction interviewers probe: "deployment != release".

## 2.3 Merge Strategies

- **Merge commit** — preserves full history and branch topology; can clutter history.
- **Squash and merge** — collapses a branch into one commit; clean history, loses granular commits.
- **Rebase and merge** — replays commits on top of target branch; linear history, rewrites SHAs (never rebase shared/public branches).

## 2.4 Commit & Tagging Conventions

- **Conventional Commits** (`feat:`, `fix:`, `chore:`) enable automated changelog and semantic-version bump generation (semantic-release).
- **Semantic Versioning (SemVer)** — MAJOR.MINOR.PATCH: major = breaking change, minor = backward-compatible feature, patch = backward-compatible bug fix.
- Git tags (`v1.4.2`) usually mark the exact commit that produced a released artifact — critical for traceability and rollback.

## 2.5 Pull Request / Merge Request Gates

Typical required checks before merge: passing CI build, minimum code coverage threshold, required approvals (CODEOWNERS), no unresolved conversations, branch up to date with target, and passing security/SAST scan. Branch protection rules enforce these in GitHub/GitLab/Bitbucket.

# 3. Jenkins Deep Dive

## 3.1 Architecture

Jenkins uses a **Controller (Master) – Agent (Slave)** model. The controller schedules builds, stores configuration, and serves the UI/API; agents (static or dynamic, e.g. Kubernetes pods, EC2, Docker containers) execute the actual build steps. Distributing work to agents keeps the controller lightweight and enables scaling and OS/tool diversity (Windows agent for .NET, Linux for everything else).

## 3.2 Declarative vs Scripted Pipeline

| Declarative | Scripted |
|---|---|
| Structured, opinionated syntax ( `pipeline { stages { stage { steps {} } } }` ) | Full Groovy DSL, imperative, maximum flexibility |
| Built-in `post` , `when` , `parameters` , `options` blocks | Requires manual try/catch/finally for error handling |
| Easier to lint, read, and maintain | Better for complex conditional/dynamic pipeline generation |

### Example Declarative Jenkinsfile

```
pipeline {
  agent { label 'linux-docker' }
  environment {
    IMAGE = "myrepo/app:${env.BUILD_NUMBER}"
  }
  options {
    timeout(time: 30, unit: 'MINUTES')
    disableConcurrentBuilds()
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') {
      steps { sh 'mvn -B clean package' }
    }
    stage('Unit Test') {
      steps { sh 'mvn test' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Build Image') {
      steps { sh "docker build -t ${IMAGE} ." }
    }
    stage('Push') {
      when { branch 'main' }
      steps {
        withCredentials([usernamePassword(credentialsId: 'docker-hub',
            usernameVariable: 'USER', passwordVariable: 'PASS')]) {
          sh "echo $PASS | docker login -u $USER --password-stdin"
          sh "docker push ${IMAGE}"
        }
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps { sh "kubectl set image deployment/app app=${IMAGE}" }
    }
  }
  post {
    failure { slackSend channel: '#builds', message: "Build ${env.BUILD_NUMBER} failed" }
    always  { cleanWs() }
  }
}
```

## 3.3 Key Concepts

- **Shared Libraries** — reusable Groovy pipeline code hosted in a separate repo, imported with `@Library('my-shared-lib') _` , to avoid duplicating pipeline logic across many Jenkinsfiles.
- **Plugins** — Jenkins' extensibility model (Git, Kubernetes, Blue Ocean, Pipeline Utility Steps, Credentials Binding). Plugin sprawl/version conflicts are a common real-world pain point.
- **Executors** — number of concurrent build slots per agent; tune based on CPU/memory to avoid resource starvation.
- **Jenkins Agents on Kubernetes** — the Kubernetes plugin spins up ephemeral pod agents per build, ensuring clean, reproducible, and

scalable build environments; pods terminate after the build (no snowflake agents).

- **Credentials Store** — centrally manages secrets (username/password, SSH keys, secret text, certificates) referenced by ID rather than hardcoded values.
- **Blue/Green Controller Upgrades** — Jenkins itself needs HA: JCasC (Jenkins Configuration as Code) plus backups (thinBackup, or shared EFS/PVC for `JENKINS_HOME`) support disaster recovery.

## 3.4 Triggers

- **SCM Polling** — Jenkins periodically checks for changes (legacy, adds latency).
- **Webhooks** — Git server (GitHub/GitLab/Bitbucket) notifies Jenkins instantly on push/PR events (preferred, low latency).
- **Cron-based** (`triggers { cron('H 2 * * *') }`) for nightly builds.
- **Upstream/downstream** job triggers — chaining pipelines.

## 3.5 Scaling & HA Considerations Senior favorite

Single Jenkins controller is a SPOF. Mitigations: run the controller on Kubernetes with a persistent volume for `JENKINS_HOME`, regular backups/snapshots, JCasC to make the controller reproducible from code, and horizontal scaling only for agents (controller itself does not horizontally scale well — this is a common gotcha interviewers probe). For very large orgs, some migrate from Jenkins to SaaS CI (GitHub Actions/GitLab CI) specifically to avoid controller-scaling and plugin-maintenance overhead — be ready to discuss this trade-off.

# 4. GitLab CI/CD

## 4.1 Core Concepts

GitLab CI/CD is configured via `.gitlab-ci.yml` in the repo root. Pipelines are made of **stages**, each containing **jobs** that run in parallel by default within a stage, and sequentially across stages. Execution happens on **GitLab Runners** (shared, group, or project-specific; executors: Docker, Shell, Kubernetes, VirtualBox).

### Example .gitlab-ci.yml

```
stages: [build, test, scan, deploy]

variables:
  IMAGE_TAG: "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"

build:
  stage: build
  image: docker:24
  services: [docker:24-dind]
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG

unit_test:
  stage: test
  image: node:20
  script: [npm ci, npm test]
  coverage: '/Coverage: \d+\.\d+%/'
  artifacts:
    reports:
      junit: junit.xml

sast:
  stage: scan
  include:
    - template: Security/SAST.gitlab-ci.yml

deploy_prod:
  stage: deploy
  environment:
    name: production
    url: https://app.example.com
  script:
    - kubectl set image deployment/app app=$IMAGE_TAG
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```

## 4.2 Key Features

- **rules / only / except** — control when jobs run; `rules` is the modern, more flexible replacement for `only/except`.
- **Environments** — track deployment history per environment, support Review Apps (dynamic environment per merge request).
- **Artifacts & Cache** — artifacts pass files between stages/jobs (e.g., build output); cache speeds up repeated jobs (e.g., `node_modules`) but is not guaranteed durable — a key interview distinction (cache = optimization, artifact = required output).
- **DAG pipelines (`needs:`)** — jobs can start as soon as their specific dependencies finish rather than waiting for the whole previous stage, reducing pipeline duration.
- **Parent-child & multi-project pipelines** — `trigger:` keyword lets one pipeline kick off another, useful for microservice/mono-repo orchestration.
- **Merge Trains** — queues merge requests and tests them cumulatively against the latest target branch state to prevent "merge and immediately break main" races under high commit volume.
- **GitLab Auto DevOps** — opinionated, zero-config pipeline templates covering build/test/scan/deploy for common stacks.

## 4.3 Runners

Runners can be *shared* (available org-wide), *group*, or *specific* (dedicated to one project, e.g. for special hardware/GPU). The Kubernetes executor spins up a pod per job (similar tradeoffs to Jenkins' Kubernetes plugin): clean, ephemeral, autoscalable, but with cold-start latency vs a warm shell runner.

# 5. GitHub Actions

## 5.1 Core Concepts

Workflows are YAML files in `.github/workflows/`. A workflow contains **jobs** (run in parallel by default, unless `needs:` is set), each with **steps** that run actions (reusable units from the Marketplace) or shell commands. Workflows are triggered by **events** (`push`, `pull_request`, `schedule`, `workflow_dispatch`, `repository_dispatch`).

### Example Workflow

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node: [18, 20]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '${{ matrix.node }}' }
      - run: npm ci
      - run: npm test
      - uses: actions/upload-artifact@v4
        with: { name: coverage, path: coverage/ }

  build-image:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: myrepo/app:${{ github.sha }}

  deploy:
    needs: build-image
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: kubectl set image deployment/app app=myrepo/app:${{ github.sha }}
```

## 5.2 Key Features

- **Reusable workflows** (`workflow_call`) vs **composite actions** — reusable workflows share entire jobs across repos; composite actions bundle multiple steps into one reusable action.
- **Matrix builds** — run the same job across a combination of OS/versions/parameters in parallel.
- **Environments & protection rules** — required reviewers, wait timers, and environment-scoped secrets gate deployments.
- **Self-hosted runners** vs GitHub-hosted — self-hosted needed for private network access, special hardware, or cost control at scale; requires patching/security ownership.
- **OIDC (OpenID Connect) to cloud providers** — workflows can assume an AWS IAM role / Azure/GCP identity without storing long-lived cloud credentials as secrets — a major modern security best practice interviewers love to probe.
- **Caching** (`actions/cache`) vs **artifacts** (`actions/upload-artifact`) — same distinction as GitLab: cache = speed optimization (best-effort), artifact = required build output retained for the run.
- **Concurrency groups** — cancel superseded runs of the same branch/PR to save compute and avoid race conditions on deploy jobs.

## 5.3 Security Considerations **Senior favorite**

- Pin third-party actions to a full commit SHA, not a mutable tag, to avoid supply-chain attacks via a compromised action.
- `pull_request_target` runs with write permissions and access to secrets even for forked-repo PRs — a classic misconfiguration that leaks secrets to untrusted code; prefer `pull_request` plus manual approval for first-time contributors.
- Scope `GITHUB_TOKEN` permissions minimally per workflow (`permissions:` block) instead of relying on the default broad token.

# 6. Other CI Tools

## 6.1 CircleCI

Config in `.circleci/config.yml`. Uses **orbs** (shareable reusable config packages, similar to GitHub Actions Marketplace actions). Has strong support for **workflows** with fan-out/fan-in job dependencies and **resource classes** to size compute per job.

## 6.2 Azure Pipelines

YAML-based (`azure-pipelines.yml`) with **stages → jobs → steps/tasks**. Strong integration with Azure Boards/Repos and enterprise-friendly **approvals & gates** on environments. **Multi-stage YAML pipelines** replaced the older Release Pipelines (classic UI-based) as the recommended approach.

## 6.3 AWS CodePipeline / CodeBuild / CodeDeploy

- **CodePipeline** — orchestrates stages (source → build → test → deploy), integrating other AWS services.
- **CodeBuild** — managed build service (buildspec.yml), scales compute on demand, no servers to manage.
- **CodeDeploy** — automates deployments to EC2/on-prem/Lambda/ECS, natively supports blue/green and canary via `appspec.yml` hooks.

## 6.4 TeamCity & Bamboo

JetBrains TeamCity and Atlassian Bamboo are older enterprise CI servers, conceptually similar to Jenkins (build agents + build configurations) but with more built-in UI/reporting and less plugin sprawl. Still common in .NET-heavy or Atlassian-stack (Jira/Bitbucket) enterprises.

## 6.5 Spinnaker

Purpose-built **continuous delivery** platform (originated at Netflix) specializing in complex, multi-cloud deployment orchestration: native support for canary analysis (via Kayenta), blue/green, and rollback across AWS/GCP/Kubernetes/Azure. Typically paired with a CI tool (Jenkins/GitHub Actions builds the artifact; Spinnaker handles the deployment pipeline).

## 6.6 Choosing a Tool — Talking Points

| Factor | Consideration |
|---|---|
| Hosting model | SaaS (GitHub Actions/CircleCI) = less ops overhead; self-hosted (Jenkins) = full control, more maintenance |
| Ecosystem | Already on GitHub → Actions; already on GitLab → GitLab CI; enterprise Azure shop → Azure Pipelines |
| Extensibility | Jenkins plugin ecosystem is unmatched but adds maintenance burden |
| Compliance/data residency | Self-hosted runners/agents needed when code/artifacts cannot leave a private network |
| Deployment complexity | Multi-cloud/canary-heavy orgs often add Spinnaker or ArgoCD on top of a simpler CI tool |

# 7. Docker & Containerization in CI/CD

## 7.1 Why Containers in CI/CD

Containers give reproducible, isolated build/test environments and guarantee "build once, run anywhere" — the same image that passed tests in CI is exactly what runs in production, eliminating environment drift.

## 7.2 Multi-Stage Builds Senior favorite

```
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Multi-stage builds keep the final image small (no build tools/compilers) and reduce attack surface. Interviewers often ask: "how do you shrink image size / reduce vulnerabilities?" — the answer is multi-stage builds + minimal/distroless base images + `.dockerignore`.

## 7.3 Image Tagging & Registries

- Never rely solely on `:latest` in production — it is mutable and non-reproducible; tag with commit SHA or semantic version, and optionally re-tag stable releases.
- **Registries:** Docker Hub, Amazon ECR, Google Artifact Registry, Azure Container Registry, JFrog Artifactory, Harbor (self-hosted, with built-in vulnerability scanning and RBAC).
- **Image promotion pattern** — build one image, push to a registry, then promote (re-tag) the same digest across dev/staging/prod rather than rebuilding per environment.

## 7.4 Docker-in-Docker (DinD) vs Socket Mounting

To build images inside a CI job you either run **DinD** (nested Docker daemon, more isolated but slower and needs privileged mode) or mount the host's `/var/run/docker.sock` (faster, shares the host daemon's cache, but breaks container isolation and is a security risk in shared/multi-tenant runners). Modern alternative: **Kaniko**, **Buildah**, or **BuildKit** rootless builders that build OCI images without a privileged daemon at all — preferred in Kubernetes-based CI for security.

## 7.5 Image Scanning

Tools: Trivy, Grype, Clair, Snyk Container, Docker Scout. Typically run as a pipeline stage right after build; pipelines can be configured to fail the build on CRITICAL/HIGH CVEs above a policy threshold.

# 8. Kubernetes, Helm & GitOps (ArgoCD/Flux)

## 8.1 Push vs Pull (GitOps) Deployment Models Senior favorite

| Push-based (traditional CI/CD) | Pull-based (GitOps) |
|---|---|
| CI pipeline runs `kubectl apply` / `helm upgrade` directly against the cluster | An in-cluster controller (ArgoCD/Flux) continuously reconciles cluster state to match a Git repo |
| CI/CD tool needs cluster credentials — larger blast radius if compromised | Cluster credentials never leave the cluster; Git is the single source of truth |
| Harder to detect/undo manual "kubectl" drift | Automatic drift detection and self-healing (reverts manual changes back to Git state) |

This push-vs-pull distinction, and specifically "why GitOps improves security posture," is one of the most common senior DevOps/SRE interview questions.

## 8.2 Helm

Helm packages Kubernetes manifests into **charts** with templated values (`values.yaml`), enabling parameterized, versioned, reusable deployments. Key commands: `helm install`, `helm upgrade --install`, `helm rollback`. Helm maintains release history in-cluster, enabling one-command rollback to a previous revision.

## 8.3 ArgoCD

- Watches a Git repo (App-of-Apps pattern for many microservices) and syncs cluster state to match.
- **Sync policies:** manual vs automated; `selfHeal` reverts manual drift; `prune` removes resources deleted from Git.
- **App health & sync status** shown in UI: Synced/OutOfSync, Healthy/Degraded/Progressing.
- Supports Helm, Kustomize, and plain YAML as templating backends.
- **ArgoCD Rollouts** extends this with canary/blue-green analysis using metrics (Prometheus) to automate progressive delivery decisions.

## 8.4 Flux

Similar GitOps controller (CNCF project), source-controller + kustomize-controller + helm-controller architecture; commonly paired with **Flagger** for automated canary/blue-green promotion based on live metrics.

## 8.5 Typical End-to-End Flow

```
Developer pushes code
  → CI builds/tests/scans, pushes image:sha to registry
  → CI updates image tag in a separate "config"/GitOps repo (or Helm values)
  → ArgoCD/Flux detects the Git change
  → Controller reconciles the cluster to match the new desired state
  → Progressive delivery controller (Argo Rollouts/Flagger) shifts traffic gradually,
     watching error-rate/latency metrics, auto-rolling back on regression
```

> Note the separation: the *application* CI pipeline never touches the cluster directly — it only updates a Git repo that GitOps tooling watches. This separation of "build" and "deploy" repos/pipelines is a strong signal of GitOps maturity in interviews.

# 9. Deployment Strategies

| Strategy | How it works | Pros | Cons |
|---|---|---|---|
| Recreate | Stop old version fully, then start new version | Simple, no version overlap | Downtime |
| Rolling Update | Gradually replace old pods/instances with new ones, a few at a time | No downtime, default in Kubernetes Deployments | Two versions run simultaneously during rollout; slow rollback |
| Blue-Green | Deploy new version (Green) alongside old (Blue) fully, then switch traffic (LB/DNS) at once | Instant rollback (switch back), full pre-prod validation on real infra | Requires 2x infrastructure during switch; DB schema compatibility needed |
| Canary | Route a small % of traffic to new version, gradually increase while monitoring metrics | Limits blast radius, real production validation with real users | Needs good traffic-shaping (service mesh/ingress) and monitoring/automation |
| A/B Testing | Route traffic based on user segment/feature for business experimentation, not just safety | Business metric validation | Not primarily a safety mechanism; often combined with canary |
| Shadow / Dark Launch | Mirror production traffic to the new version without returning its response to the user | Zero user-facing risk, tests real load | Complex to set up; side-effects (writes) must be handled carefully |

## 9.1 Rollback Strategies Senior favorite

- **Kubernetes:** `kubectl rollout undo deployment/app` reverts to the previous ReplicaSet.
- **Helm:** `helm rollback <release> <revision>`.
- **Blue-Green:** flip the router/load balancer back to Blue — fastest possible rollback (seconds).
- **Database migrations:** the hardest part of rollback — always write *backward-compatible* migrations (expand/contract pattern: add new column, dual-write, migrate reads, then drop old column in a later release) so a code rollback never leaves the schema broken.
- **Feature flags:** instantly disable a feature without a full deployment rollback — often faster than any pipeline-based rollback.

## 9.2 Progressive Delivery Automation

Tools like **Argo Rollouts** and **Flagger** automate canary analysis: they query Prometheus/Datadog for error rate, latency (p99), and saturation during each traffic-shift step, and automatically pause/rollback if metrics breach defined thresholds — removing the need for a human to babysit the rollout.

# 10. Infrastructure as Code (Terraform, Ansible, CloudFormation)

## 10.1 Terraform in CI/CD

```
stages: [validate, plan, apply]

validate:
  script: [terraform fmt -check, terraform validate]

plan:
  script:
    - terraform init
    - terraform plan -out=tfplan
  artifacts:
    paths: [tfplan]

apply:
  script: terraform apply -auto-approve tfplan
  when: manual
  environment: production
```

- **Remote state** (S3+DynamoDB lock, Terraform Cloud, GCS) is mandatory in CI — local state cannot be shared safely across pipeline runs/parallel jobs.
- **State locking** prevents concurrent applies from corrupting state.
- **Plan gating** — always review a `plan` output (often posted as a PR comment via tools like Atlantis) before a manual `apply` approval, especially for prod.
- **Workspaces / directory-per-environment** — separate state per environment (dev/staging/prod) to avoid a bad plan touching the wrong environment.
- **Drift detection** — scheduled `terraform plan` jobs detect manual out-of-band changes.

## 10.2 Ansible in CI/CD

Ansible is often used for configuration management/orchestration (post-provisioning steps, application config, orchestrated multi-host deploys) rather than resource provisioning. Runs playbooks against inventory from a CI job or AWX/Ansible Tower/Ansible Automation Platform for RBAC and scheduling at scale.

## 10.3 AWS CloudFormation / CDK

CloudFormation defines AWS resources declaratively in JSON/YAML **stacks**; **Change Sets** preview changes before applying (analogous to `terraform plan`). AWS CDK lets you author infrastructure in general-purpose languages (TypeScript/Python) that synthesize to CloudFormation templates.

## 10.4 Policy as Code

Tools like **Open Policy Agent (OPA)/Conftest**, **Checkov**, and **tfsec** scan IaC for misconfigurations (open security groups, unencrypted storage, overly broad IAM) as an automated pipeline gate before `apply` — shifting security/compliance checks left into the IaC review stage.

# 11. Artifact & Dependency Management

## 11.1 Why a Dedicated Artifact Repository

Build outputs (jars, npm packages, Docker images, Python wheels, Helm charts) should be stored in a versioned, immutable, access-controlled repository — not just left on a CI runner's disk. Common tools: **JFrog Artifactory**, **Sonatype Nexus**, **GitHub Packages/GitLab Package Registry**, cloud-native registries (ECR, GAR, ACR).

## 11.2 Key Practices

- **Build once, promote many** — the exact binary/image tested in CI is what gets promoted through environments, never rebuilt.
- **Immutable versions** — once published, a version number/tag is never overwritten (prevents "silent" behavior changes under an existing version).
- **Dependency caching/proxying** — Artifactory/Nexus can proxy public registries (npm, Maven Central, PyPI) so builds aren't dependent on public registry uptime, and to scan cached dependencies for vulnerabilities (SCA).
- **Retention policies** — clean up old snapshot/dev artifacts to control storage cost, while keeping all release artifacts.

# 12. Secrets Management

## 12.1 Anti-Patterns

- Hardcoding secrets in source code or pipeline YAML.
- Storing secrets in plaintext environment variables printed to build logs.
- Sharing one long-lived credential across all pipelines/environments.

## 12.2 Correct Patterns

- **Native CI secret stores** — Jenkins Credentials, GitHub Actions Secrets/Environments, GitLab CI/CD Variables (masked + protected).
- **Dedicated secrets managers** — HashiCorp Vault (dynamic, short-lived secrets, encryption-as-a-service), AWS Secrets Manager/Parameter Store, Azure Key Vault, GCP Secret Manager.
- **OIDC federation** — CI/CD assumes a short-lived cloud role via OIDC token exchange instead of storing static cloud access keys at all (see GitHub Actions section 5.2).
- **Kubernetes secrets** should be encrypted at rest (KMS provider) and ideally never stored raw in Git — tools like **Sealed Secrets** or **External Secrets Operator** (syncing from Vault/Secrets Manager into K8s Secrets at runtime) solve the "secrets in GitOps repo" problem.
- **Rotation** — automate credential rotation; dynamic secrets (Vault) issue short-lived, auto-expiring credentials per pipeline run rather than static long-lived ones.

# 13. DevSecOps & Pipeline Security

## 13.1 Security Scan Types Across the Pipeline

| Type | What it checks | When it runs | Example tools |
|---|---|---|---|
| SAST (Static Application Security Testing) | Source code for insecure patterns | On every commit/PR, early | SonarQube, Semgrep, Checkmarx, CodeQL |
| SCA (Software Composition Analysis) | Known CVEs in open-source dependencies | On build/dependency change | Snyk, OWASP Dependency-Check, Dependabot |
| Container/Image Scanning | OS packages & layers in the built image | After image build | Trivy, Grype, Clair |
| IaC Scanning | Misconfigurations in Terraform/K8s manifests | Before apply | Checkov, tfsec, OPA/Conftest |
| DAST (Dynamic Application Security Testing) | Running application for exploitable vulnerabilities | Against a deployed test/staging environment | OWASP ZAP, Burp Suite |
| Secret Scanning | Committed credentials/API keys | Pre-commit and in CI | Gitleaks, TruffleHog, GitHub secret scanning |

## 13.2 Shift-Left Security

Moving these checks as early as possible (pre-commit hooks, PR-time SAST) is cheaper than catching issues at DAST/production stage — cost and blast radius of a vulnerability both increase the later it's found. Interviewers commonly ask you to design a "secure pipeline" combining several of the above at the right stage.

## 13.3 Supply Chain Security

- **SBOM (Software Bill of Materials)** — a manifest of every component/dependency in an artifact (formats: SPDX, CycloneDX), generated automatically during build for compliance and rapid vulnerability triage (e.g., after a Log4Shell-type incident, quickly find every affected artifact).
- **Artifact signing & provenance** — **Sigstore/Cosign** signs container images; **SLSA framework** defines provenance levels attesting how/where an artifact was built, so consumers/clusters can verify an image actually came from the expected CI pipeline before running it (admission controllers like Kyverno/OPA Gatekeeper can enforce "only run signed images").
- **Least privilege** — scope every pipeline's tokens/roles/credentials to only what that specific job needs.
- **Pipeline integrity** — protect the pipeline definition itself with branch protection/required review, since a compromised Jenkinsfile/workflow file is equivalent to compromising production.

# 14. Monitoring, Observability & Feedback Loops

## 14.1 Pipeline Observability

- Track build duration, queue time, flaky-test rate, and pipeline success rate as engineering metrics, not just application metrics.
- Centralize CI/CD logs (ELK/Loki) so agent/runner failures are searchable across historical runs, not just the last build's console output.

## 14.2 Deployment & Post-Deploy Monitoring

- **Prometheus + Grafana** — metrics collection and dashboards; commonly the data source that progressive-delivery tools (Argo Rollouts/Flagger) query to auto-gate a canary.
- **ELK/EFK Stack (Elasticsearch, Logstash/Fluentd, Kibana)** or **Loki+Grafana** — centralized log aggregation.
- **Distributed tracing** (Jaeger, Zipkin, OpenTelemetry) — essential in microservices to see the true end-to-end effect of a new deployment across service boundaries.
- **Synthetic monitoring / smoke tests** — automated post-deploy checks (hit key endpoints, verify health checks) run immediately after deploy as the pipeline's final gate before declaring success.
- **Alerting** (Alertmanager, PagerDuty, Opsgenie) — ties back into incident response; a deployment that trips an alert should be correlated automatically with "what changed" (deployment markers/annotations in Grafana are a common practice).

## 14.3 Closing the Loop

Mature pipelines annotate dashboards with deployment events so the first question during an incident — "what changed recently?" — can be answered in seconds. This is frequently framed as an interview scenario: "production error rate spiked at 2:14pm, how do you find out if it's a deploy?"

# 15. SRE Concepts for CI/CD

## 15.1 SLI / SLO / SLA

| Term | Definition |
|---|---|
| SLI (Indicator) | A measured metric, e.g. request success rate, latency p99 |
| SLO (Objective) | Internal target for an SLI, e.g. "99.9% of requests succeed over 30 days" |
| SLA (Agreement) | External, often contractual commitment to a customer, usually looser than the internal SLO, with penalties for breach |

## 15.2 Error Budgets & Release Gating Senior favorite

An error budget is `1 - SLO` (e.g., a 99.9% SLO gives a 0.1% error budget over the period). SRE practice ties CI/CD velocity to the error budget: while budget remains, teams can deploy freely/aggressively; once the budget is exhausted, releases are throttled or frozen and effort shifts to reliability work until the budget recovers. This is the clearest mechanism connecting SRE philosophy directly to CI/CD pipeline policy — expect this exact question in SRE interviews.

## 15.3 Toil Reduction via CI/CD

SRE defines **toil** as manual, repetitive, automatable operational work with no enduring value. CI/CD pipelines are a primary toil-reduction mechanism: automated testing/deployment removes manual release toil; self-service pipelines let developers deploy without filing an Ops ticket.

## 15.4 Incident Management Integration

- **Deployment freeze** during active incidents or high-risk periods (e.g., Black Friday) — pipelines should support an emergency "pause all deploys" switch.
- **Automated rollback on alert** — some pipelines integrate with monitoring to auto-trigger a rollback if key SLIs regress post-deploy (see Argo Rollouts/Flagger in section 9.2).
- **Postmortems / blameless RCA** — a failed deployment causing an incident should produce a written root-cause analysis and concrete pipeline/process action items (e.g., "add this check as an automated gate").
- **Runbooks** — pipeline failures and rollback steps should be documented/automated so on-call doesn't need tribal knowledge at 3am.

## 15.5 Change Management & Risk

Best practice: bucket every deployment into a change risk tier (low/medium/high) based on blast radius; low-risk changes get fully automated pipelines, high-risk changes (schema migrations, config affecting all tenants) get extra manual approval gates, canary duration, and stakeholder notification.

# 16. Common Errors & Troubleshooting Playbook

**Error:** Pipeline fails intermittently ("flaky" build/test) with no code change.

**Troubleshoot:** Check for test order dependency/shared state, race conditions in async tests, timing-sensitive assertions, insufficient container resource limits causing OOM/timeouts, or non-deterministic external calls (network, real time/date). Fix: isolate test data per test, add retries only for genuinely external flakiness (not code bugs), quarantine flaky tests with a tracking ticket rather than ignoring, and increase timeouts/resources where legitimately needed.

**Error:** `docker: permission denied while trying to connect to the Docker daemon socket.`

**Fix:** The build agent's user is not in the `docker` group, or the socket isn't mounted into the CI container. Add the runner user to the docker group, mount `/var/run/docker.sock`, or switch to a daemonless builder (Kaniko/Buildah) to avoid needing the Docker daemon at all.

**Error:** Jenkins job stuck in queue — "Waiting for next available executor".

**Fix:** No agent matches the required label, all executors are busy, or a node is offline/disconnected. Check agent labels match exactly, scale up agents (or enable Kubernetes dynamic agent provisioning), and check node connectivity/Java version mismatches between controller and agent.

**Error:** `CrashLoopBackOff` immediately after a Kubernetes deployment from the pipeline.

**Fix:** `kubectl describe pod` and `kubectl logs --previous` to see the actual crash reason (missing env var/config, failing readiness/liveness probe, OOMKilled, bad entrypoint). Common root causes: a ConfigMap/Secret not updated in the same release, a probe timeout too aggressive for app startup time, or an image tag that didn't actually change (deployment "succeeded" but pulled a stale cached image due to `imagePullPolicy` misconfiguration).

**Error:** Deployment "succeeds" but users still see the old version.

**Fix:** Classic caching/rollout issues: CDN/browser cache not invalidated, old pods still receiving traffic during a slow rolling update (check `readinessProbe` and `maxUnavailable/maxSurge`), a stale load balancer target group, or (in GitOps) ArgoCD stuck "OutOfSync"/"Progressing" due to a sync hook failure — check `argocd app get` for the actual sync/health status rather than trusting the CI pipeline's "success" alone.

**Error:** Pipeline works locally/in one environment but fails in CI ("works on my machine").

**Fix:** Environment drift — different tool versions, missing env vars, OS-specific paths/line endings, or a dependency resolved differently (no lockfile committed). Fix: pin exact tool versions (via Dockerfile/.tool-versions/asdf), commit lockfiles (package-lock.json, poetry.lock), and always build inside the same container image used in production.

**Error:** `Error: secret "my-secret" not found` / pipeline fails pulling credentials.

**Fix:** Wrong secret scope (project vs group vs org level), secret not injected into the right environment/branch protection rule, or a masked variable being referenced with the wrong name. Verify variable scoping rules for the specific CI tool (e.g., GitLab protected variables only inject on protected branches).

**Error:** Terraform apply fails with `Error acquiring the state lock`.

**Fix:** A previous run crashed/was killed without releasing the DynamoDB/state lock. Verify no other apply is genuinely running, then force-unlock with `terraform force-unlock <lock-id>` (with caution — only after confirming no concurrent run).

**Error:** Build passes but artifact/image is much larger than expected, or vulnerability scan suddenly fails on a previously-passing base image.

**Fix:** Base image drift (a `:latest`-style floating tag pulled a new, larger, or newly-CVE'd version). Fix: pin base images by digest, and add scheduled scans/rebuilds so CVEs in the base image are caught proactively rather than only at build time.

**Error:** Merge queue/pipeline bottleneck — long queue times as team scales.

**Fix:** Parallelize test suites (split by shard/tag), cache dependencies aggressively, move slow E2E tests to a separate post-merge pipeline rather than blocking every PR, add more runners/agents (autoscaling), and use DAG-based (`needs:`) job graphs instead of purely sequential stages.

## 16.1 General Debugging Method (State This in Interviews)

1. Read the actual failure log/message first — don't guess.
2. Reproduce locally or in an isolated debug pipeline run if possible.
3. Check "what changed" — code diff, dependency versions, infra config, base images, pipeline YAML itself.
4. Check resource constraints (CPU/memory/disk/network) on the runner/agent/pod.
5. Isolate: does it fail on every branch/environment or just one?
6. Fix the root cause, then add a regression guard (test, lint rule, or pipeline gate) so it can't silently reoccur.

# 17. Tricky Questions & Traps

**Q: What's the difference between Continuous Delivery and Continuous Deployment? (Most candidates get this wrong)**

Both automate everything up through "release-ready", but Delivery keeps a manual approval before production, Deployment removes it entirely. The trap: many candidates say they're the same thing, or reverse the definitions.

**Q: If your pipeline is "green" (all tests passed), is it safe to deploy?**

Not necessarily — green only means the checks you wrote passed. It says nothing about untested edge cases, infra capacity, an exhausted SRE error budget, or a concurrent incident freeze. Senior answer: pipeline success is necessary but not sufficient; deployment decisions should also weigh error budget status, change risk tier, and current incident state.

**Q: Should you rebuild the artifact for each environment (dev/staging/prod)?**

No — build once, promote the same immutable artifact through environments. Rebuilding per environment risks a different artifact reaching prod than the one that was tested (different dependency resolution, different base image pulled at a different time), defeating the purpose of testing.

**Q: Is cache the same as an artifact in GitLab CI / GitHub Actions?**

No. Cache is a best-effort speed optimization (may be evicted/missing, pipeline must still work without it); artifacts are the required, guaranteed output of a job that downstream jobs/stages depend on.

**Q: A canary shows zero errors after 5 minutes at 5% traffic — is it safe to go to 100%?**

Trap question — 5 minutes at 5% traffic may not have exercised rare code paths, batch jobs, end-of-day/end-of-month logic, or enough statistical volume to detect a real regression. Senior answer: canary duration and traffic percentage should scale with (a) statistical significance for your traffic volume and (b) the periodicity of the risky code paths (e.g. run at least one full billing cycle for a billing change).

**Q: Does using GitOps (ArgoCD) mean you no longer need a CI pipeline?**

No — GitOps tools (ArgoCD/Flux) handle the CD/reconciliation half only; you still need a CI pipeline to build, test, scan, and produce the versioned artifact/image and update the GitOps repo/manifest. GitOps replaces "push-based deploy", not the whole CI process.

**Q: If a rollback is instant, do you still need backward-compatible database migrations?**

Yes — a fast application rollback is useless if the database schema has already moved forward incompatibly. This is why the expand/contract migration pattern matters: the schema must support both old and new code simultaneously during any rollout/rollback window.

**Q: Is a self-hosted CI runner always more secure than a SaaS one?**

Trap — not automatically. Self-hosted runners bring full control but also full responsibility for patching, isolation between jobs (a compromised job on a shared self-hosted runner can pivot to others), and secret handling. SaaS runners are ephemeral and provider-patched but you trust the vendor's isolation. The "more secure" answer depends on your team's ability to operate the runner infrastructure correctly.

**Q: You mounted the Docker socket into your CI job to build images — any concern?**

Yes — this effectively grants root on the host to anything running in that CI job (container escape / access to every other container on the host). Prefer rootless builders (Kaniko/Buildah/BuildKit rootless) especially on shared multi-tenant runners.

**Q: 100% code coverage — does that mean the pipeline guarantees no bugs?**

No — coverage measures lines executed by tests, not correctness of assertions or business-logic edge cases; you can have 100% coverage with weak/no assertions. Interviewers use this to see if you conflate coverage with quality.

**Q: A hotfix needs to skip the full pipeline to go out immediately — good idea?**

Generally no — skipping tests/scans to save time is exactly how hotfixes cause a second incident. Better answer: maintain a fast-tracked but still-automated hotfix pipeline (smaller, prioritized test subset + mandatory smoke test) rather than a fully manual bypass.

# 18. Expected Interview Q&A (Basic → Advanced)

## 18.1 Basic Level

**Q: What is a CI/CD pipeline?**

An automated sequence of stages (build, test, scan, package, deploy) that takes code from a commit to a running, verified release with minimal manual intervention.

**Q: What is a build artifact?**

The compiled/packaged output of a build stage (jar, binary, Docker image, npm package) that gets versioned, stored in a repository, and promoted through environments unchanged.

**Q: What's the difference between a webhook trigger and polling in CI?**

A webhook is pushed instantly by the Git server when an event occurs (low latency, event-driven); polling has the CI server periodically check for changes (adds delay, wastes resources on empty checks).

**Q: What is a Dockerfile and why is it used in CI/CD?**

A text file with instructions to build a container image layer by layer; used in CI/CD to produce a reproducible, portable artifact that behaves identically across dev/test/prod.

**Q: What is idempotency and why does it matter in pipelines?**

Running the same operation multiple times produces the same result as running it once. It matters because pipelines/deployments may be retried after partial failure; non-idempotent steps (e.g., "insert row" without a check) can corrupt state on retry.

**Q: What is a build agent/runner?**

A machine (VM, container, or Kubernetes pod) that actually executes pipeline steps, separate from the CI server/controller that schedules and orchestrates the work.

**Q: Name the main stages of a typical pipeline.**

Source → build → unit test → static analysis/security scan → package/artifact → deploy to test/staging → further tests → deploy to production → post-deploy verification.

## 18.2 Intermediate Level

**Q: How would you speed up a slow CI pipeline?**

Parallelize independent jobs/test shards, cache dependencies, use a DAG ( `needs:` ) instead of strictly sequential stages, right-size/scale runners, split slow E2E suites out of the PR-blocking path, and use incremental builds where the tool supports it.

**Q: How do you manage configuration differences between environments?**

Externalize config (env vars, ConfigMaps, parameter store) from the artifact itself; the same immutable artifact is deployed everywhere, with only its configuration/values (Helm values.yaml, environment variables) changing per environment.

**Q: What is the difference between a rolling update and a blue-green deployment?**

Rolling update gradually replaces instances in-place (no extra infra, but rollback is gradual and two versions run simultaneously); blue-green runs a full duplicate environment and switches traffic atomically (instant rollback, but double the infra during the switch).

**Q: How do you handle secrets in a pipeline securely?**

Use the CI tool's native masked/protected secret store or an external secrets manager (Vault/Secrets Manager), never hardcode in code/YAML, scope secrets to the minimum environment/branch needed, and prefer short-lived/OIDC-federated credentials over static long-lived ones.

**Q: Explain the difference between SAST and DAST.**

SAST analyzes source code statically (no running app) for insecure patterns early in the pipeline; DAST tests a running deployed application from the outside (like an attacker would), catching runtime/config issues SAST can't see, typically later in the pipeline (staging).

**Q: What is a feature flag and how does it relate to CI/CD?**

A runtime toggle that decouples code deployment from feature release — incomplete or risky code can be merged and deployed to production behind a disabled flag, then enabled later (instantly, without a new deployment) once ready or validated for a subset of

users.

**Q: What happens if a Kubernetes deployment's readiness probe is misconfigured?**

If too aggressive/short, healthy pods get marked not-ready and traffic is prematurely cut or the rollout stalls; if too lenient, traffic gets routed to pods that aren't actually ready to serve, causing errors during rollout.

**Q: What's the purpose of a staging environment if you already have good tests?**

Staging validates integration with real (or near-real) dependencies, infra-level config, network policies, and performance/load characteristics that unit/mocked tests can't fully capture — it's the last check before real user traffic.

**Q: How do you prevent a bad deployment from taking down the whole system?**

Progressive delivery (canary/blue-green) to limit blast radius, automated health-check gates and rollback, circuit breakers/timeouts at the service level, and a deployment freeze policy tied to error budget/incident status.

## 18.3 Advanced / Senior Level

**Q: Design a CI/CD pipeline for a microservices architecture with 50+ services.**

Key points to hit: independent per-service pipelines (not one monolithic pipeline) triggered on changes to that service's path/repo; shared reusable pipeline templates (shared libraries/reusable workflows) to avoid duplication; a separate GitOps config repo per environment (or per cluster) that CI updates with new image tags, decoupled from the app CI; centralized artifact registry with strict tagging/promotion; standardized SLO/observability instrumentation baked into a common base image/sidecar so every service is consistently monitorable; contract testing between services to catch breaking API changes before deploy; and a change-management/canary policy that scales per service's criticality tier.

**Q: How do you enforce that only artifacts that passed your pipeline can run in production?**

Sign artifacts at the end of a successful pipeline run (Cosign/Sigstore), attach provenance metadata (SLSA), and enforce signature verification at the cluster admission layer (OPA Gatekeeper/Kyverno policy) so unsigned or externally-built images are rejected outright — this closes the gap between "CI says it's safe" and "the cluster actually only runs what CI approved".

**Q: How would you reduce Mean Time to Restore (MTTR) using your pipeline?**

Fast, one-command rollback (blue-green flip, Helm rollback, feature flag kill switch), deployment markers on dashboards for instant correlation with incidents, small/frequent deploys to shrink the diff you have to reason about during an incident, automated canary analysis to catch regressions before full rollout, and rehearsed runbooks so on-call doesn't improvise under pressure.

**Q: Your org wants to move from monthly releases to multiple deploys per day. What changes across the org, not just tooling?**

Trunk-based development + feature flags to enable frequent small merges; automated (not manual) QA gates since manual regression testing can't scale to that cadence; a shift from big-bang change-review meetings to automated policy-as-code gates; on-call/ops process changes to handle more frequent (but smaller, safer) changes; and cultural buy-in that deploy != release, reducing fear around shipping incomplete features behind flags.

**Q: How do you handle a multi-region/multi-cluster deployment in your pipeline?**

Typically GitOps per cluster (App-of-Apps pattern in ArgoCD, or Flux per cluster), a staged rollout order (canary region first, then wave out globally), region-aware feature flags/config, and cross-region health-check gating so a bad deploy in region A halts propagation to regions B/C automatically.

**Q: How do you balance developer velocity against pipeline security gates that slow things down?**

Push expensive/slow checks (DAST, full E2E) to run in parallel or post-merge rather than blocking every PR; make fast checks (lint, unit test, SAST for changed files only) the PR-blocking gate; use risk-tiering so low-risk changes get lighter gates and high-risk changes get the full gauntlet; and continuously measure gate false-positive rates so security checks don't become "ignored noise".

**Q: What is the "build once, deploy many" principle and why do teams violate it despite knowing better?**

Principle: produce a single immutable artifact/image once, then promote the identical artifact across environments by only changing configuration. Teams violate it under time pressure (rebuilding "quickly" for prod without going through the pipeline), environment-specific compiled config baked into the image instead of externalized, or legacy tooling that doesn't support artifact promotion — all of which reintroduce "works in staging, breaks in prod" risk.

**Q: How would you design rollback for a change that involves both a service deployment and a database migration?**

Use the expand/contract pattern: migrations must be additive/backward-compatible (new nullable column, dual writes) so that the previous application version continues to function correctly against the new schema; only after the new version is fully rolled out and stable do you run a later, separate migration to remove old columns/tables. This guarantees the fast application-level rollback is never blocked by an incompatible schema.

**Q: How do you decide when a CI/CD pipeline needs a human approval gate versus full automation?**

Base it on blast radius and reversibility: fully reversible, low-blast-radius, well-tested changes (typical app code with good canary automation) can be fully automated; changes with high blast radius, compliance implications, or difficult/slow reversibility (schema changes, pricing logic, multi-tenant config, security policy changes) should keep a manual approval gate, ideally from someone other than the author.

# 19. System Design Style Questions

These are open-ended "whiteboard" questions common at senior DevOps/SRE/Cloud Ops interviews. Structure every answer as: **clarify requirements → propose architecture → call out trade-offs → mention monitoring/rollback/security.**

**Q: Design a CI/CD pipeline for a company migrating from monolith to microservices.**

Talk through: transitional period needs both monolith and service pipelines coexisting; strangler-fig pattern means new services get independent pipelines immediately while the monolith pipeline shrinks over time; shared platform team owns pipeline templates/golden paths so 50 teams aren't reinventing CI config; centralized observability standard from day one so cross-cutting incidents (monolith calling a new service) are debuggable.

**Q: Design a zero-downtime deployment system for a stateful service (e.g., a database-backed API with WebSocket connections).**

Talk through: rolling update with connection draining (graceful shutdown hooks, `preStop` lifecycle hook, SIGTERM handling with a grace period), session affinity or externalizing session state (Redis) so any pod can serve any request, readiness probes that only pass once a pod is truly ready to accept new connections, and careful handling of in-flight WebSocket connections during pod termination (drain gracefully rather than hard-kill).

**Q: Design a CI/CD system that must support both cloud and on-prem/air-gapped deployments.**

Talk through: build artifacts once in a cloud CI, then push to a mirrored/offline-syncable artifact registry reachable by air-gapped sites; avoid baking cloud-specific assumptions (IAM roles, cloud metadata service calls) into the base image; a separate, simplified deployment mechanism for air-gapped sites (e.g., a signed artifact bundle + manual/scripted import) since GitOps controllers need network access to a Git remote, which air-gapped environments may lack, requiring a local Git mirror.

**Q: Design an incident-safe deployment pipeline for a payments system.**

Talk through: strict canary with statistically significant traffic and full billing-cycle-aware soak time, mandatory dual approval for production deploys, feature flags for any risk-bearing logic, idempotent payment operations (idempotency keys) so retries during a rollback never double-charge, and an explicit deployment freeze calendar around high-traffic periods, combined with strong audit logging for compliance (PCI-DSS).

# 20. Behavioral / Experience-Based Questions

For a 5-year profile, interviewers expect concrete stories (use the STAR method: Situation, Task, Action, Result). Prepare one real example for each of the following prompts from your own project history:

- "Tell me about a time a bad deployment caused an incident. What did you do, and what changed afterward?"
- "Describe a CI/CD pipeline you designed or significantly improved from scratch. What was slow/broken before, what did you change, and what was the measurable impact (deploy frequency, lead time, failure rate)?"
- "Tell me about a disagreement with a developer or team about a pipeline gate/policy (e.g., they wanted to skip a security scan). How did you resolve it?"
- "Describe the most complex production rollback you've executed. What made it complex, and how did you make the decision to roll back versus fix forward?"
- "How have you mentored or influenced other engineers to adopt better CI/CD practices?"
- "Tell me about a time you had to balance delivery deadlines against pipeline/security best practices."

> Senior/SRE interviews often weight these as heavily as technical questions — have 4-6 stories ready and be able to quantify outcomes (e.g., "reduced deployment time from 45 min to 8 min", "cut change failure rate from 20% to 6%").

# 21. Quick Reference Cheat Sheet

## 21.1 Useful Commands

```
# Kubernetes
kubectl rollout status deployment/app
kubectl rollout undo deployment/app
kubectl rollout history deployment/app
kubectl describe pod <pod>
kubectl logs <pod> --previous

# Helm
helm upgrade --install app ./chart -f values-prod.yaml
helm rollback app 3
helm history app

# ArgoCD
argocd app sync my-app
argocd app get my-app
argocd app rollback my-app

# Docker
docker build -t app:sha .
docker scan app:sha        # or: trivy image app:sha
docker system prune -af    # reclaim disk on runners

# Terraform
terraform plan -out=tfplan
terraform apply tfplan
terraform force-unlock <lock-id>
terraform state list

# Git
git tag -a v1.4.2 -m "release"
git log --oneline --graph --all
```

## 21.2 One-Line Definitions to Have Ready

| | |
|---|---|
| **Idempotent** | Same result no matter how many times you run it |
| **Immutable infrastructure** | Never modify running infra/artifacts in place — replace them |
| **Blue-Green** | Two full environments, instant traffic switch, instant rollback |
| **Canary** | Gradual traffic shift with live metric-based validation |
| **GitOps** | Git as source of truth; a cluster-side controller pulls and reconciles |
| **SLI/SLO/SLA** | Measured metric / internal target / external contractual commitment |
| **Error budget** | Allowed unreliability (1 - SLO) that gates release velocity |
| **Toil** | Manual, repetitive, automatable ops work with no lasting value |
| **SBOM** | Full inventory of components/dependencies in an artifact |
| **Shift-left** | Move testing/security checks earlier in the pipeline |
| **DORA metrics** | Deploy frequency, lead time, change failure rate, MTTR |

## 21.3 Day-Before-Interview Checklist

- Re-read section 15 (SRE concepts) — the single most differentiating section for senior/SRE roles.
- Rehearse 2-3 STAR stories out loud (section 20).
- Be ready to draw a pipeline diagram on a whiteboard/shared doc from memory (section 1.4).
- Review the tricky Q&A (section 17) — these are designed to catch shallow knowledge.
- Know the exact tools used in your own recent projects well enough to go deep if asked a follow-up.

End of CI/CD Interview Bible — Good luck!